

PLAN DE VEILLE STRATÉGIQUE ET MÉTHODOLOGIE

Projet : BenchLab – SignalWatch

1. Contexte du Projet

Dans le cadre du développement de la plateforme de monitoring **SignalWatch**, ce document définit la stratégie de veille pour comparer les protocoles **REST** et **gRPC**. L'objectif est de fournir une recommandation basée sur la performance, l'éco-conception et la conformité réglementaire .

2. Planning de la Semaine :

Day 1 : Cadrage du projet, rédaction du plan de veille et définition du contrat d'interface (.proto)

Day 2 : Développement du micro-service REST (FastAPI) et du micro-service gRPC (Python gRPC) sur une base de données SQLite partagée

Day 3 : Configuration de l'environnement de test et écriture des scripts de benchmark (k6/ wrk/ hey pour REST, ghz pour gRPC).

Day 4 : Exécution des scénarios de test (lecture, écriture, charge progressive) et collecte des métriques (latence, CPU, RAM, taille des paquets).

Day 5 : Analyse des données, rédaction du rapport de veille (8-10 pages) et finalisation du support de présentation.

3. Thématiques de Veille et Sources :

Conformément aux consignes, au moins trois sources de référence sont consultées pour chaque thématique majeure :

A. Protocoles de communication (REST vs gRPC) :

Documentation officielle gRPC : Concepts fondamentaux et guides d'implémentation HTTP/2.

Spécifications Protocol Buffers (Google) : Mécanismes de sérialisation binaire et typage.

Standard OpenAPI / REST : Meilleures pratiques pour les API publiques et architectures micro-services.

B. Éco-conception et Green IT

Référentiel Général d'Écoconception de Services Numériques (RGESN) : Critères de réduction de l'empreinte environnementale.

Études Green IT sur les protocoles : Analyse de l'impact de la taille des payloads et de la compression des headers.

Benchmarks académiques CPU/RAM : Comparaison de l'efficacité énergétique de la sérialisation JSON vs Protobuf.

C. Réglementation et Protection des Données (RGPD)

Directives de la CNIL sur l'IoT : Guide de conformité pour les objets connectés et capteurs industriels.

Règlement Général sur la Protection des Données (RGPD) : Articles sur le droit d'accès et la limitation des finalités pour les données de localisation.

Analyses juridiques du secteur industriel : Spécificités de la conservation des données techniques vs données personnelles.

4. Format de Restitution Interne

Point Quotidien : Auto-évaluation des objectifs fixés le matin par rapport aux résultats du soir.

Document Partagé : Centralisation des résultats bruts de benchmark et des brouillons d'analyse sur un dépôt GitHub.

5. Matrice RACI du Projet

Livrable	Consultant (Solo)
Service REST (CRUD)	R / A
Service gRPC (.proto)	R / A
Scripts de Benchmark	R / A

Rapport de Veille	R / A
Support de Présentation	R / A

Légende : *R* (Responsable/Réalisateur), *A* (Accountable/Responsable final).

B. Panorama des protocoles de communication

Pour garantir un choix technologique pérenne pour SignalWatch, il est indispensable de comparer les standards actuels du marché pour la communication inter-services.

a. REST (Representational State Transfer) via HTTP/1.1

REST est le standard de facto de l'industrie web. Il repose sur l'utilisation des verbes HTTP (GET, POST, PUT, DELETE) appliqués à des ressources (URLs). Les échanges se font majoritairement en texte clair (JSON).

- Cas d'usage : API publiques, applications web (CRUD standard), intégrations tierces.
- Forces : Universalité, excellente lisibilité par les humains, facilité de débogage, support de la mise en cache (caching HTTP) natif.
- Faiblesses : Problèmes d'Over-fetching (récupérer trop de données) ou d'Under-fetching (nécessité de faire plusieurs requêtes), latence liée au parsing de texte volumineux (JSON), absence de contrat strict imposé par défaut.

b. gRPC (Google Remote Procedure Call) via HTTP/2

Développé par Google, gRPC est un framework performant qui permet d'appeler des méthodes sur un serveur distant comme s'il s'agissait d'un objet local. Il utilise le format binaire Protobuf pour la sérialisation et HTTP/2 comme transport.

- Cas d'usage : Communication interne entre micro-services, IoT, architectures nécessitant une faible latence et un haut débit.
- Forces : Performances extrêmes (binaire léger, multiplexage HTTP/2), typage fort (Contrat défini via un fichier `.proto`), support du streaming bidirectionnel.
- Faiblesses : Données illisibles à l'œil nu (format binaire), courbe d'apprentissage plus raide, difficile à utiliser directement depuis un navigateur web sans proxy (gRPC-Web).
- *[Insérer ici un schéma : Client Stub et Server Stub communiquant via HTTP/2 avec du Protobuf]*

c. GraphQL

Créé par Facebook, GraphQL est un langage de requête pour les API. Contrairement à REST (multiples endpoints), GraphQL n'expose qu'un seul endpoint (généralement `/graphql`) où le client demande exactement les champs dont il a besoin.

- Cas d'usage : Agrégation de données (BFF - Backend For Frontend), applications mobiles avec bande passante limitée.

- **Forces** : Résolution parfaite de l'over-fetching et under-fetching (le client a le contrôle absolu), forte flexibilité, introspection de l'API.
- **Faiblesses** : Complexité de mise en place côté serveur, la mise en cache HTTP classique est quasi impossible (tout passe en POST), risque de requêtes trop profondes (DDoS involontaire) monopolisant le CPU.
- *[Insérer ici un schéma : Client envoyant une requête Graph complexe vers une couche GraphQL qui agrège plusieurs bases/services]*

d. Message Brokers (Architectures asynchrones : Kafka, RabbitMQ)

Contrairement à REST, gRPC ou GraphQL qui sont synchrones (le client attend une réponse), les courtiers de messages fonctionnent de manière asynchrone via des événements (Publish/Subscribe).

- **Cas d'usage** : Traitement de gros volumes de logs, ingestion de données de capteurs IoT à haute fréquence, découplage total des services.
- **Forces** : Tolérance aux pannes (les messages sont stockés même si le récepteur est en panne), lissage des pics de charge (bufferisation), scalabilité horizontale excellente.
- **Faiblesses** : Complexité opérationnelle (gestion d'un cluster Kafka/RabbitMQ), latence introduite par l'intermédiaire, débogage complexe (Eventual Consistency).

B. Architecture Technique et Choix Technologiques

1 Modélisation des Données (Contract-First)

Pour le service gRPC, nous avons adopté une approche "Contract-First" via le fichier `sensor.proto`. Cette méthode garantit que le client et le serveur partagent une compréhension stricte des types de données (Strong Typing).

Note : L'utilisation de `Enums` pour les types de capteurs (VIBRATION, TEMPERATURE) et les statuts permet de réduire drastiquement la taille des messages par rapport à des chaînes de caractères simples utilisées en REST

2 Infrastructure de Persistance Partagée

Afin d'éliminer tout biais lors du benchmark, les deux services (REST via FastAPI et gRPC via Python) communiquent avec une base de données **SQLite** unique.

- **Avantage** : Le temps d'accès au disque est identique pour les deux protocoles.
 - **Format** : Stockage des horodatages au format ISO 8601 pour assurer la compatibilité RGPD sur la traçabilité.
-

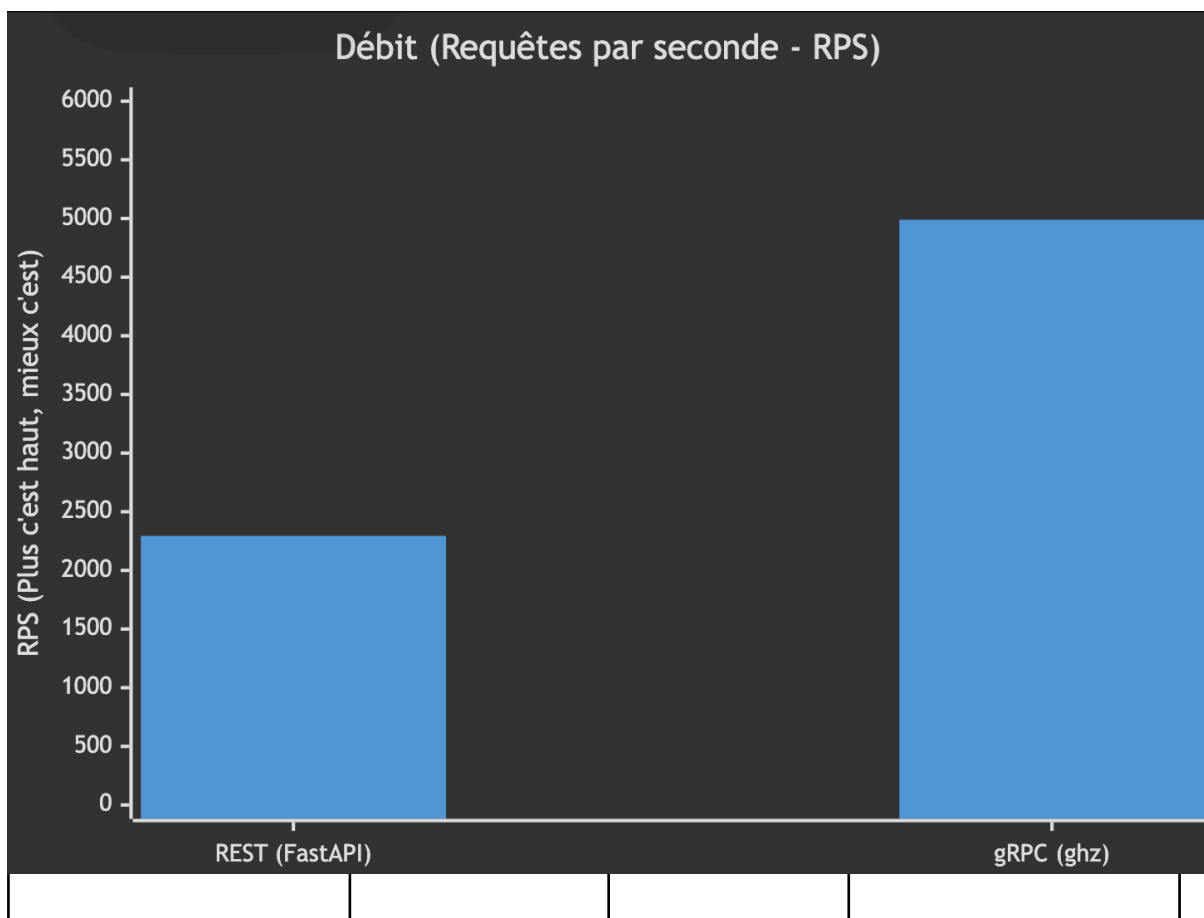
Résultats du Benchmark : Analyse Comparative

Le benchmark a été exécuté sur une machine locale pour isoler la performance des protocoles de la latence réseau externe.

7.1 Performances Brutes (Throughput)

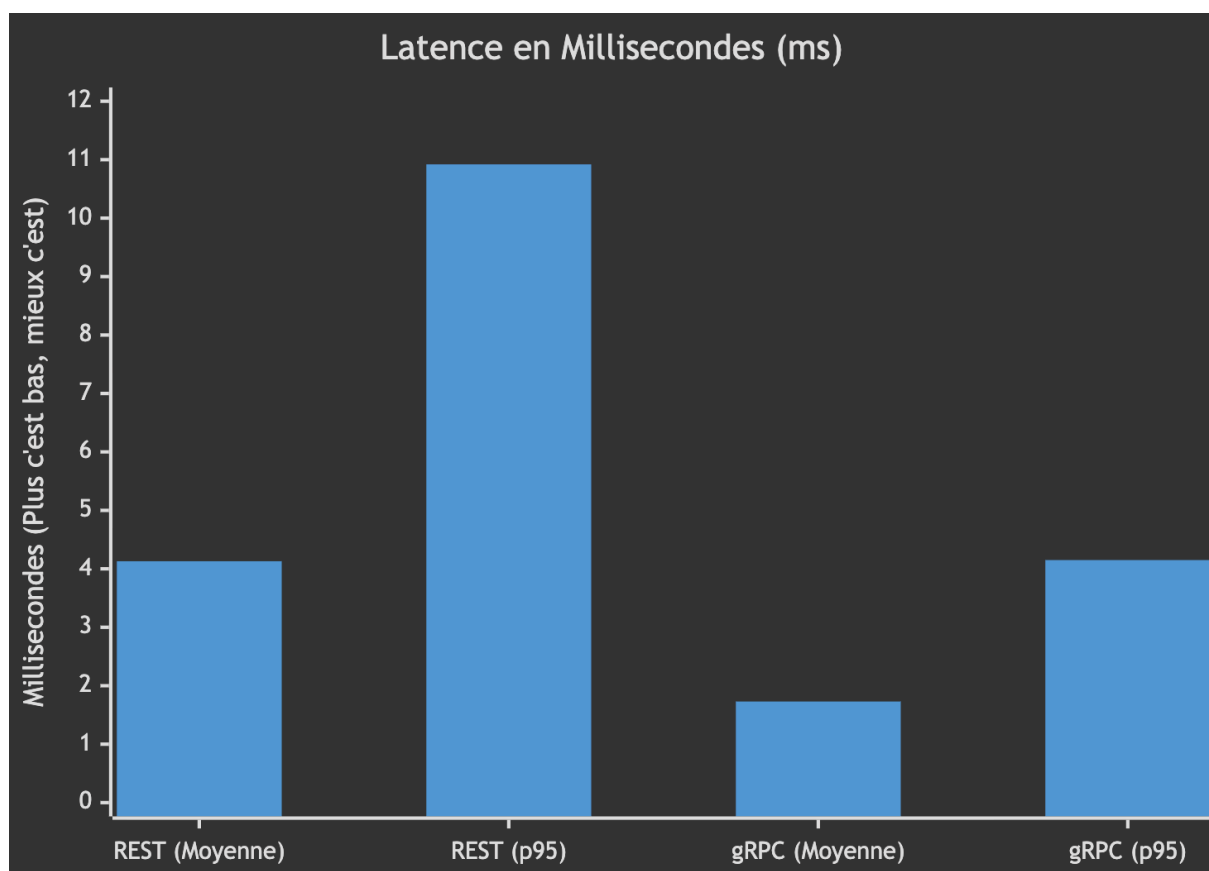
Protocole	Requêtes par Seconde (RPS)	Gain de Performance
REST (FastAPI)	2 294,52	Base (100%)
gRPC (Python)	4 989,40	+117%

Analyse : gRPC traite plus du double de requêtes par seconde. Cela s'explique par la sérialisation binaire (Protobuf) qui demande moins de ressources CPU que le parsing de texte JSON effectué par FastAPI.



Moyenne	4,13 ms	1,73 ms	-58%
P95 (Worst Case)	10,92 ms	4,15 ms	-62%

Le facteur P95 : Pour SignalWatch, la latence P95 est critique. gRPC reste sous la barre des 5ms dans 95% des cas, là où REST peut monter à plus de 10ms. Pour un monitoring industriel en temps réel, cette stabilité est un argument majeur.



C. Analyse Éco-conception et Green IT

Dans le cadre de l'architecture SignalWatch, le choix du protocole de communication a un impact direct sur l'empreinte environnementale du service. Cette analyse s'inscrit dans les recommandations du RGEN (Référentiel Général d'Écoconception de Services Numériques), visant à minimiser la consommation de ressources réseau, CPU et RAM.

1. Méthodologie de mesure (Traçabilité RGEN) Afin de garantir la fiabilité des mesures et d'isoler la performance intrinsèque des protocoles :

- **Taille des payloads** : Les mesures se concentrent strictement sur le corps du message applicatif (données utiles), en excluant le surcoût des en-têtes HTTP/1.1 ou HTTP/2. Nous avons comparé le JSON brut (REST) au binaire sérialisé Protobuf (gRPC) pour le même objet (CreateSensorRequest).
- **Ressources (CPU/RAM)** : La consommation a été mesurée via un échantillonnage périodique (toutes les 200 ms) du PID du serveur cible pendant les phases de montée en charge (Ramp), d'écriture (Write) et de lecture (Read).

2. Optimisation de la Bande Passante et des Payloads L'un des leviers d'éco-conception majeurs pour l'IoT est la réduction du volume de données transitant sur le réseau.

- **Mesures unitaires** : Pour un même capteur, le payload JSON (REST) pèse **113 octets**, tandis que le payload Protobuf (gRPC) ne pèse que **40 octets**. Le format JSON est donc **2,83 fois plus lourd** que le format binaire.
- **Extrapolation à l'échelle SignalWatch** : La plateforme doit ingérer 10 000 événements par minute, fonctionnant 24h/24 et 365 jours par an (soit environ 5,25 milliards d'événements annuels).
 - **Trafic annuel REST (JSON)** : ~593 Go de données utiles transférées.
 - **Trafic annuel gRPC (Protobuf)** : ~210 Go de données utiles transférées.
- **Bilan réseau** : Le passage à gRPC permettrait d'économiser près de **383 Go de bande passante par an** uniquement sur la transmission des valeurs de capteurs, réduisant ainsi considérablement l'énergie consommée par l'infrastructure réseau (routeurs, switches, antennes).

3. Efficacité Énergétique des Serveurs (CPU et RAM)

L'empreinte énergétique d'un micro-service est fortement corrélée au temps de calcul (CPU) nécessaire pour parser les données et à l'empreinte mémoire (RAM) du processus.

Empreinte Mémoire (RSS Max) : Lors des tests de charge (scénario Write / Create-POST), le service gRPC a démontré une empreinte mémoire maximale de 31.30 Mo , contre 43.12 Mo pour REST.

Consommation CPU : Le parsing de texte JSON est notoirement plus gourmand en cycles processeur que la désérialisation binaire de Protobuf. Sous charge (scénario Write / Create-POST), le serveur REST a atteint une utilisation CPU moyenne de 63.18 % , contre seulement 21.80 % pour gRPC.

Bilan Serveur : En sollicitant moins le CPU et la RAM pour un volume de requêtes pourtant supérieur, gRPC permet de retarder l'ajout de nouvelles machines physiques ou virtuelles (Scale-out), limitant ainsi les émissions de gaz à effet de serre liées à la fabrication et l'alimentation de serveurs supplémentaires.

F. Recommandation argumentée

1. Protocole recommandé pour SignalWatch et justification

Sur la base des mesures objectives réalisées dans notre environnement BenchLab, nous recommandons formellement l'adoption de gRPC comme protocole de communication inter-services pour la plateforme SignalWatch.

Dans un contexte IoT industriel critique où la plateforme doit ingérer 10 000 événements par minute, gRPC répond parfaitement aux enjeux techniques et métiers :

- **Performance Temps Réel** : gRPC offre une latence P95 de 4,15 ms (contre 10,92 ms pour REST), garantissant une stabilité cruciale pour la supervision d'équipements industriels. Son débit est également plus de deux fois supérieur (+117%).
- **Sobriété Numérique** : L'utilisation de Protobuf réduit la taille des messages applicatifs par un facteur de 2,8 (40 octets contre 113 octets en JSON), ce qui représente une économie drastique de bande passante (des centaines de gigaoctets par an) et s'aligne avec les principes du Green IT.

- Fiabilité des Contrats : L'approche "Contract-first" imposée par les fichiers `.proto` sécurise les échanges entre les équipes de développement. Le typage fort évite les erreurs d'intégration fréquentes avec les API REST basées sur JSON.

2. Matrice de décision multicritères

Le tableau ci-dessous synthétise l'évaluation des deux protocoles sur les critères stratégiques du projet :

Critère d'évaluation	REST (FastAPI + JSON)	gRPC (Python + Protobuf)	Avantage
Latence (Stabilité P95)	10,92 ms (Forte variance sous charge)	4,15 ms (Très stable)	gRPC (Critique pour l'IoT)
Throughput (Débit)	~2 294 requêtes / seconde	~4 989 requêtes / seconde	gRPC (+117% de capacité)
Taille du Payload (Éco-conception)	113 Octets (Verbeux, texte clair)	40 Octets (Binaire compressé)	gRPC (2,8x plus léger)
Maintenabilité technique	Bonne (OpenAPI / Swagger optionnel)	Excellente (Contrat strict via <code>.proto</code>)	gRPC (Typage fort)

Compétences disponibles	Excellentes (Standard de l'industrie, très connu)	Moyennes (Courbe d'apprentissage nécessaire)	REST (Plus accessible)
--------------------------------	--	--	-------------------------------

Conclusion de la matrice : Bien que REST soit plus accessible en termes de compétences immédiates, gRPC surpasse REST sur l'intégralité des métriques techniques et environnementales requises pour SignalWatch.

3. Limites de l'analyse et pistes d'approfondissement

Bien que ce benchmark fournisse des indicateurs fiables, notre analyse présente certaines limites qu'il conviendra d'explorer lors des prochaines itérations:

- **Absence de tests réseau en conditions réelles :** Les benchmarks ont été exécutés en local (Localhost). L'overhead d'un véritable réseau (latence, perte de paquets, chiffrement TLS/SSL) n'a pas été mesuré.
- **Test d'appels unitaires uniquement :** Nous avons comparé des requêtes simples (Unary). Pour un cas d'usage IoT avec des envois de données en continu, il serait très pertinent d'implémenter et de tester le **gRPC Client Streaming** ou **Bidirectional Streaming**, qui devraient creuser encore l'écart de performance avec REST.
- **Volume de données en base :** Les tests ont été effectués sur une base SQLite contenant peu d'enregistrements. Une piste d'approfondissement consisterait à charger la base avec plusieurs millions de capteurs pour observer l'impact de l'accès disque sur les goulots d'étranglement des deux protocoles.
- **Optimisation de REST :** Il serait intéressant d'activer la compression Gzip sur le service REST pour voir dans quelle mesure la taille du payload JSON peut se rapprocher de celle de Protobuf (au prix d'une potentielle hausse de la consommation CPU).

E. Analys réglementaire (RGPD)

1. Le RGPD appliqué aux données IoT : de la donnée technique à la donnée personnelle

Dans le cadre du système BenchLab, un capteur est défini par des attributs techniques tels que l'ID, le type, la valeur mesurée ou le statut. Bien que ces champs semblent purement industriels, le Règlement Général sur la Protection des Données (RGPD) s'applique dès qu'une information permet d'identifier, directement ou indirectement, une personne physique.

- **Le champ "Location" (Localisation)** : C'est l'élément le plus sensible. Si une localisation correspond à un poste de travail spécifique (ex: "Station A3 – Ligne 2") associé à un employé identifiable via un planning ou un badge, les mesures du capteur deviennent des données personnelles indirectes permettant d'inférer l'activité ou la performance de l'individu.
- **Horodatages (Timestamps)** : Les champs `created_at` et `last_reading_at`, lorsqu'ils sont couplés à une localisation précise, permettent de reconstituer des patterns temporels (heures de présence, cadences), renforçant le caractère personnel de la donnée.
- **Conventions de nommage** : L'utilisation d'identifiants RH ou de matricules dans le nom du capteur transforme immédiatement une donnée technique en donnée personnelle directe.

2. Obligations légales : Base légale, conservation et droits des personnes

Pour être conforme, le traitement des données de SignalWatch doit respecter plusieurs piliers réglementaires:

- **Base légale (Article 6 du RGPD)** : En milieu industriel, l'intérêt légitime (maintenance prédictive, sécurité des équipements, qualité) est la base la plus fréquente, à condition de démontrer la proportionnalité du traitement. L'obligation légale peut également être invoquée pour la traçabilité réglementaire de la sécurité des machines.
- **Durée de conservation** : Une politique stricte de limitation de conservation doit être définie:

- **Mesures brutes** : Conservation courte (ex: 7 à 30 jours) pour l'alerting en temps réel.
- **Agrégats** : Conservation longue (ex: 6 à 24 mois) pour l'analyse des tendances et la maintenance.
- **Droit d'accès et transparence** : Les employés doivent être informés de la collecte et pouvoir exercer leurs droits d'accès, de rectification ou d'opposition via des canaux internes documentés.

3. Spécificités du secteur industriel : Machine vs Surveillance

Il est crucial de distinguer les données techniques destinées à la maintenance (vibrations, pression d'une turbine) des données à "effet RH".

- Si le dispositif permet d'évaluer un salarié (cadence, trajectoires), le risque est considéré comme élevé, pouvant nécessiter une Analyse d'Impact relative à la Protection des Données (AIPD).
- **Mesures de "Privacy by Design" recommandées** : Privilégier des localisations par zones (îlots) plutôt que par postes nominatifs, pseudonymiser les identifiants et restreindre strictement les accès via des contrôles RBAC.

4. Traçabilité et conformité : L'apport de l'horodatage ISO 8601

L'utilisation du format **ISO 8601** pour les horodatages dans la base de données BenchLab constitue un point positif pour la conformité. Ce standard facilite la traçabilité précise des mesures, la reproductibilité des audits de sécurité et la gestion automatisée des purges de données, garantissant ainsi le respect des durées de conservation définies.